

Лабораторна робота №6

Тема: Архітектура Feature-Sliced Design (FSD), керування станом за допомогою Pinia та використання UI-бібліотеки компонентів.

Мета: ознайомлення з методологією Feature-Sliced Design для організації структури фронтенд-застосунку, з централізованим керуванням станом у Pinia та з принципами інтеграції зовнішньої бібліотеки UI-компонентів.

Вимоги до звіту:

1. Лабораторна робота повинна бути виконана **в окремій гілці** feature/fsd у vue-lottery-app github репозиторію студента. Гілку створити від коду лабораторної роботи №5 (feature/routing).
2. Функціонал застосунку vue-lottery-app з лабораторної №5 (маршрутизація, користувачі, авторизація, Repository) має зберегтися.
3. Історія комітів має відображати кроки виконання (див. розділ «Порядок виконання»). Один коміт «refactor» з усіма змінами не приймається.
4. Дотримання TypeScript-типізації;
5. Використання Composition API;
6. Під час написання застосунку обов'язково дотримуватись [style guide](#);
7. Використовуйте scss. З UI-бібліотеки підключайте лише потрібні компоненти та стилі.
8. Використовуйте ESLint. Правила імпортів між шарами FSD мають перевірятися лінтером (Steiger або eslint-plugin-boundaries).
9. Адреса API та інші налаштування середовища зберігаються у **.env** файлі.

Порядок виконання (мінімальна історія комітів):

1. `chore: add Pinia, UI library, FSD linter` - встановлення залежностей;
2. `refactor: introduce FSD folder structure` - створення шарів і перенесення файлів без зміни логіки;
3. `refactor: split Repository across FSD layers - shared/api, entities/user/api`, реєстр сервісів;
4. `feat: add Pinia stores - session та users`;
5. `feat: add shared/ui wrappers` - обгортки над UI-бібліотекою;
6. `refactor: use wrappers and stores in pages and widgets`;
7. `chore: fix FSD lint errors`.

Практична частина

Самостійна робота студента:

1. [Feature-Sliced Design: Overview](#);
2. [FSD: Layers](#);
3. [FSD: Slices and segments](#);
4. [FSD: Public API](#);
5. [Steiger - лінтер для FSD](#);
6. [Pinia Core Concepts](#) (Defining a Store, State, Getters, Actions);
7. [Pinia: Using a store outside of a component](#);
8. [Navigation Guards](#);
9. документація обраної UI-бібліотеки: розділи Installation, Theming, Tree-shaking, Auto Import тощо.

Теоретична частина

Feature-Sliced Design

Feature-Sliced Design (FSD) - архітектурна методологія для організації коду фронтенд-застосунків. Вона задає єдині правила того, **де розміщувати код та які модулі можуть залежати від яких**. Це робить структуру проекту передбачуваною, спрощує навігацію по коду, масштабування й командну роботу.

Код у FSD ділиться на три рівні організації: **шари (layers)**, **слайси (slices)** та **сегменти (segments)**.

Шари розташовані від найвищого до найнижчого:

Шар	Призначення
app	Все, що робить застосунок працюючим: ініціалізація, роутер, провайдери (Pinia, UI-бібліотека), глобальні стилі, реєстрація сервісів.
pages	Сторінки застосунку (один слайс - одна сторінка, зазвичай відповідає маршруту).
widgets	Великі самодостатні блоки інтерфейсу, що складаються з features та entities (шапка сайту, таблиця користувачів).
features	Дії користувача, що мають бізнес-цінність (вхід у систему, створення користувача, пошук).
entities	Бізнес-сутності предметної області (user, session): типи, сервіси, store.
shared	Перевикористовуваний код без прив'язки до бізнес-логіки: HTTP-клієнт, UI-кіт, утиліти, конфіг.

Шари **app** та **shared** не діляться на слайси, а складаються безпосередньо із сегментів.

Слайси - це розбиття всередині шару за предметною областю (наприклад, **entities/user**, **features/auth**, **pages/users**).

Сегменти - розбиття слайса за технічним призначенням. Типові назви:

- **ui** - компоненти;
- **model** - стан, типи, бізнес-логіка (сюди належать Pinia store);
- **api** - взаємодія із зовнішніми джерелами даних (сюди належать репозиторії/сервіси);
- **lib** - допоміжні функції слайса;
- **config** - константи та конфігурація.

Головні правила залежностей:

1. Модуль може імпортувати лише з шарів, що розташовані **нижче** за нього (наприклад, **features** може імпортувати з **entities** і **shared**, але не з **widgets**).
2. Слайси одного шару **не імпортують один одного** (**features/auth** не може імпортувати з **features/user-form**).
3. Слайс відкриває назовні лише **Public API** - файл **index.ts**, що реекспортує дозволені частини. Імпорт «вглиб» слайса (**entities/user/model/store**) заборонений.

Приклад Public API слайса:

```
// src/entities/user/index.ts
export type { User, CreateUserDto, UpdateUserDto } from './model/types'
export { UserService } from './api/user.service'
export { useUsersStore } from './model/users.store'
```

Приклад структури застосунку:

```
src/
├── app/
│   ├── providers/
│   │   ├── router/           # createRouter, guard-и
│   │   ├── services.ts      # реєстрація сервісів у ServiceProvider
│   │   └── ui.ts             # підключення UI-бібліотеки
│   ├── styles/
│   ├── App.vue
│   └── main.ts
├── pages/
│   ├── home/
│   ├── about/
│   ├── users/
│   ├── user-details/
│   ├── login/
│   └── not-found/
```



Repository у структурі FSD

Патерн Repository з лабораторної №5 зберігається, але його складові розподіляються по шарах:

Частина	Розташування
HTTP-клієнт, інтерфейси <code>Readable</code> , <code>Creatable</code> , <code>Editable</code> , <code>Deletable</code>	<code>shared/api</code>
<code>User</code> , <code>CreateUserDto</code> , <code>UpdateUserDto</code>	<code>entities/user/model</code>
<code>UserService</code>	<code>entities/user/api</code>
<code>ServiceProvider</code> (реєстр сервісів)	<code>shared/api</code>
Реєстрація <code>UserService</code> та <code>AuthService</code> у <code>ServiceProvider</code>	<code>app/providers</code>

Зверніть увагу: `ServiceProvider` з лабораторної №5 імпортує `UserService` напряму. У FSD шар `shared` не може залежати від `entities`, тому провайдер перетворюється на **універсальний реєстр**, а конкретні сервіси реєструються на рівні `app` (шар `app` може імпортувати все):

```
// src/shared/api/service-provider.ts
type Factory<T> = () => T

export class ServiceProvider {
  private static factories = new Map<string, Factory<unknown>>()
  private static instances = new Map<string, unknown>()

  static register<T>(key: string, factory: Factory<T>): void {
    this.factories.set(key, factory)
  }
}
```

```

static get<T>(key: string): T {
  if (!this.instances.has(key)) {
    const factory = this.factories.get(key)
    if (!factory) throw new Error(`Service "${key}" is not registered`)
    this.instances.set(key, factory())
  }
  return this.instances.get(key) as T
}
}

```

```

// src/app/providers/services.ts
import { ServiceProvider } from '@shared/api'
import { UserService } from '@entities/user'

export function registerServices(): void {
  ServiceProvider.register('users', () => new UserService())
}

```

Pinia

Pinia - офіційна бібліотека керування станом для Vue 3. Store - це реактивний об'єкт, доступний з будь-якого компонента, що зберігає спільний стан застосунку.

Основні поняття:

- **state** - реактивні дані (**ref**);
- **getters** - обчислювані значення (**computed**);
- **actions** - функції, що змінюють стан (у тому числі асинхронні).

Рекомендований для цієї роботи підхід - **setup stores**, які використовують Composition API:

```

// src/entities/user/model/users.store.ts
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'
import { ServiceProvider } from '@shared/api'
import type { UserService } from '../api/user.service'
import type { User, CreateUserDto, UpdateUserDto } from '../types'

export const useUsersStore = defineStore('users', () => {
  const users = ref<User[]>([])
  const total = ref(0)
  const isLoading = ref(false)
  const error = ref<string | null>(null)

  const hasUsers = computed(() => users.value.length > 0)

  const service = () => ServiceProvider.get<UserService>('users')

```

```

    async function fetchUsers(limit = 10, skip = 0) {
      isLoading.value = true
      error.value = null
      try {
        const data = await service().getAll({ limit, skip })
        users.value = data.users
        total.value = data.total
      } catch (e) {
        error.value = (e as Error).message
      } finally {
        isLoading.value = false
      }
    }

    async function removeUser(id: number) {
      await service().delete(id)
      // Оскільки DummyJSON лише імітує видалення, то оновлюємо локальний стан
      // самостійно
      users.value = users.value.filter((u) => u.id !== id)
    }

    return { users, total, isLoading, error, hasUsers, fetchUsers, removeUser }
  })

```

Використання в компоненті. Для збереження реактивності при деструктуризації state та getters використовується **storeToRefs**, а actions деструктуризуються напямую:

```

<script setup lang="ts">
import { onMounted } from 'vue'
import { storeToRefs } from 'pinia'
import { useUsersStore } from '@entities/user'

const store = useUsersStore()
const { users, isLoading, error } = storeToRefs(store)
const { fetchUsers } = store

onMounted(() => fetchUsers())
</script>

```

Store авторизації (сесії) зберігає токени та поточного користувача і синхронізується з **localStorage**:

```

// src/entities/session/model/session.store.ts
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

const STORAGE_KEY = 'session'

```

```
export const useSessionStore = defineStore('session', () => {
  const accessToken = ref<string |
  null>(localStorage.getItem(`${STORAGE_KEY}:token`))
  const isAuthenticated = computed(() => accessToken.value !== null)

  function setSession(token: string) {
    accessToken.value = token
    localStorage.setItem(`${STORAGE_KEY}:token`, token)
  }

  function clearSession() {
    accessToken.value = null
    localStorage.removeItem(`${STORAGE_KEY}:token`)
  }

  return { accessToken, isAuthenticated, setSession, clearSession }
})
```

Store можна використовувати й поза компонентами (наприклад, у navigation guard), але лише після того, як Pinia встановлено в застосунку:

```
// src/app/providers/router/guards.ts
import type { Router } from 'vue-router'
import { useSessionStore } from '@entities/session'

export function setupGuards(router: Router): void {
  router.beforeEach((to) => {
    const session = useSessionStore()
    if (to.meta.requiresAuth && !session.isAuthenticated) {
      return { name: 'login', query: { redirect: to.fullPath } }
    }
  })
}
```

UI-бібліотеки компонентів

UI-бібліотека надає готові компоненти (кнопки, поля, таблиці, діалоги, сповіщення), тематизацію та доступність, що пришвидшує розробку. Для Vue 3 поширені PrimeVue, Naive UI, Element Plus, Vuetify, Quasar, Ant Design Vue, а також підхід «copy-paste» із shadcn-vue та Tailwind CSS.

При виборі та інтеграції слід звертати увагу на:

1. **Tree-shaking** - підключаються лише використані компоненти (імпорт окремих компонентів або auto-import), щоб не збільшувати розмір бандлу;
2. **Тематизація** - можливість налаштувати кольори та типографіку під застосунок;
3. **Залежність від бібліотеки** - використання компонентів бібліотеки напряму в усьому коді ускладнює її заміну.

Для зменшення залежності компоненти бібліотеки **обгортаються** в тонкі власні компоненти, які розміщуються в **shared/ui**. Решта застосунку використовує лише **AppButton**, **AppInput** тощо, тому заміна бібліотеки зачіпає один шар:

```
<!-- src/shared/ui/AppButton/AppButton.vue (приклад на PrimeVue) -->
<script setup lang="ts">
import Button from 'primevue/button'

defineProps<{
  label: string
  loading?: boolean
  severity?: 'secondary' | 'danger'
}>()

defineEmits<{ click: [event: MouseEvent] }>()
</script>

<template>
  <Button
    :label="label"
    :loading="loading"
    :severity="severity"
    @click="$emit('click', $event)"
  />
</template>
```

Public API шару **shared/ui**:

```
// src/shared/ui/index.ts
export { default as AppButton } from './AppButton/AppButton.vue'
export { default as AppInput } from './AppInput/AppInput.vue'
export { default as AppTable } from './AppTable/AppTable.vue'
export { default as AppConfirmDialog } from './AppConfirmDialog/AppConfirmDialog.vue'
```

Завдання:

Використовуючи застосунок **vue-lottery-app** (результат лабораторної №5), виконати наступне:

- Виконати самостійну роботу студента.
- Створити гілку **feature/fsd** від **feature/routing**.
- **Структура FSD**. Реорганізувати проєкт відповідно до шарів **app**, **pages**, **widgets**, **features**, **entities**, **shared**:
 - Налаштувати alias **@** → **src** (у **vite.config.ts** та **tsconfig**);
 - **app** - **main.ts**, **App.vue**, роутер із guard-ами, реєстрація сервісів, підключення Pinia та UI-бібліотеки, глобальні стилі;

- **pages** - Home, About, Users, UserDetails, Login, NotFound;
 - **widgets** - **AppHeader** (разом з **AppNavigation** та **Logo**), **UsersTable**;
 - **features** - **auth** (форма входу, кнопка Logout), **user-form** (створення й редагування користувача з валідацією VeeValidate);
 - **entities** - **user** (типи, **UserService**, store), **session** (store, токени);
 - **shared** - HTTP-клієнт, інтерфейси **Repository**, **ServiceProvider**, UI-обгортки, конфіг.
- Кожен слайс повинен мати файл **index.ts** (Public API); імпорти між слайсами та шарами виконуються **лише** через нього.
 - Налаштувати лінтер правил FSD (Steiger або **eslint-plugin-boundaries**) так, щоб порушення напряму залежностей та глибокі імпорти були помилками. Додати прп-скрипт **lint:fsd**. Усі помилки лінера мають бути виправлені.
 - **Repository** у FSD. Перенести складові патерну Repository згідно з таблицею в теоретичній частині. Перетворити **ServiceProvider** на універсальний реєстр (без імпортів з **entities**), а реєстрацію **UserService** і **AuthService** винести в **app/providers/services.ts**.
 - **Pinia**. Підключити Pinia та реалізувати:
 - **useSessionStore** (токени, ознака авторизації, методи **setSession** / **clearSession**) - замінити попереднє зберігання стану авторизації;
 - **useUsersStore** (список, загальна кількість, **isLoading**, **error**, дії **fetchUsers**, **fetchUser**, **createUser**, **updateUser**, **removeUser**);
 - використати setup stores та **storeToRefs**;
 - після створення, редагування та видалення користувача оновлювати стан у store (DummyJSON імітує запис).
 - Компоненти сторінок і віджетів не повинні звертатися до сервісів напряму - лише через стори.
 - **UI-бібліотека**. Обрати одну бібліотеку (рекомендовано PrimeVue; допускається Naive UI або Element Plus; інша - за погодженням з викладачем) та:
 - підключити лише необхідні компоненти (auto-import або імпорт окремих компонентів);
 - створити в **shared/ui** обгортки: **AppButton**, **AppInput** (з відображенням помилки валідації), **AppTable**, **AppConfirmDialog** (підтвердження видалення користувача);
 - використати обгортки на сторінках і у віджетах, жодного прямого імпорту компонентів бібліотеки поза **shared/ui** та **app/providers/ui.ts**;
 - використати компонент сповіщень (toast/message) для результату операцій (користувача створено, оновлено, видалено, помилка авторизації);
 - налаштувати тему (основний колір) відповідно до стилю застосунку.
 - Інтегрувати VeeValidate з обгортками **AppInput** у формах входу та користувача.
 - Переконавшись, що функціонал лабораторної №5 працює без регресій: навігація, **/users/:id**, пагінація й пошук через query-параметри, авторизація, захист маршрутів, сторінка 404.

- Додати перемикач світлої/темної теми (Store налаштувань у `entities` або `features`, збереження у `localStorage`).
- Додати плагін збереження стану Pinia (`pinia-plugin-persistedstate`) і порівняти з ручною реалізацією у `session.store.ts`.
- Додати діаграму залежностей шарів проєкту до `README.md` (наприклад, у Mermaid).

Контрольні запитання

1. Що таке Feature-Sliced Design і які проблеми вона вирішує?
2. Які шари виділяються у FSD? У якому порядку вони розташовані і чому?
3. Яке головне правило імпорту між шарами? Чи може `features` імпортувати з `widgets`?
4. Чому слайси одного шару не повинні імпортувати один одного? Як вирішувати ситуацію, коли двом слайсам потрібен спільний код?
5. Що таке слайс і сегмент? Які сегменти використовуються найчастіше?
6. Що таке Public API слайса і навіщо потрібен `index.ts`?
7. Чим відрізняються шари `entities` та `features`? Наведіть приклади із цієї лабораторної.
8. Чому шари `app` та `shared` не діляться на слайси?
9. Чому `ServiceProvider` з лабораторної №5 не можна залишити в `shared` в початковому вигляді і як це було вирішено?
10. Що таке Pinia і чим вона відрізняється від звичайного composable зі спільним `ref`?
11. Чим відрізняються options stores та setup stores? Для чого потрібна функція `storeToRefs`?
12. Чому не можна напяму деструктуризувати state зі Store без `storeToRefs`?
13. Чому звертатися до сервісів (репозиторіїв) із компонентів не рекомендується, а доступ краще здійснювати через Store?
14. Для чого UI-компоненти бібліотеки обгортають у власні компоненти в `shared/ui`?
15. Що таке tree-shaking і як забезпечити підключення лише потрібних компонентів UI-бібліотеки?
16. Які переваги й недоліки FSD для невеликого застосунку, як `vue-lottery-app`?